



TP5 Network_QoS_Monitor

Desarrollo de Aplicaciones Móbiles

Profesor: Ernesto Ledesma

Alumno: Saenz Juan

Año lectivo: 2026

Documentación del Trabajo Práctico 5

Arquitectura de la Aplicación

El proyecto se diseñó utilizando un patrón de arquitectura en capas estrictamente separado para aislar el procesamiento de red de la interfaz de usuario, garantizando que el hilo principal (UI Thread) no se bloquee durante mediciones pesadas.

Las capas implementadas son:

Native Bridge (Kotlin/Android): Módulo nativo construido específicamente para comunicarse con el API de hardware de Android (`TelephonyManager`). Expone métodos hacia JavaScript para recuperar el nombre del operador celular, el tipo de red exacta y la potencia de señal (RSSI en dBm) en tiempo real.

Measurement Engine:

- PingEngine: Orquesta sondas de latencia de bajo nivel usando Sockets TCP puros.
- ThroughputEngine: Orquesta el test de descarga y subida contra un servidor propio de referencia, manejando payloads de 5MB y calculando la velocidad (MB/s).
- NetworkObserver: Listener reactivo (basado en `NetInfo`) que detecta cambios de interfaz (WiFi vs Celular).

Persistence Layer (WatermelonDB): Base de datos SQLite reactiva y de alto rendimiento. Elegida para persistir historiales de mediciones sin generar bloqueos (cuellos de botella) en el frontend.

Geo Layer: Se integra con el servicio de geolocalización nativa para obtener coordenadas GPS precisas antes de cada medición, permitiendo cruzar métricas de red con ubicación espacial.

Presentation Layer (UI): Desarrollada 100% en React Native CLI. Incluye pantallas de Medición en Tiempo Real, Historial de la Sesión, y Mapas de Calor (Heatmaps) con marcadores renderizados mediante `react-native-maps`.

Backend de Referencia: Servidor liviano (Express / Node.js) levantado en un contenedor/entorno local con túneles (Ngrok) para asegurar una prueba de red controlada (Download/Upload).

Decisiones de Diseño y Tecnologías Clave

Uso de Sockets para Latencia: En lugar de medir la latencia haciendo un request HTTP genérico (`fetch()`) que añadiría tiempos muertos de resolución DNS y parseo de cabeceras, se optó por abrir Sockets TCP (`react-native-tcp-socket`) contra hosts conocidos (8.8.8.8, 1.1.1.1). Esto garantiza que el "RTT" (Round Trip Time) medido sea un reflejo real del estado de transporte de la red.

Redondeo y Lógica Matemática de Jitter: El Jitter se calcula matemáticamente como el promedio de las diferencias absolutas entre latencias consecutivas (variación del retardo). Las métricas como pérdida de paquetes se redondean para evitar desbordamientos visuales (ej. 33.333% a 33%).

Gráficos Puros (Flexbox): Se experimentó con librerías externas para los gráficos históricos, pero para garantizar la estabilidad móvil y los FPS, se optó por desarrollar un sistema de gráficos de barras puro utilizando componentes nativos de React (`View` y `Flexbox`). Esto eliminó dependencias pesadas y posibles crasheos de renderizado.

Manejo de Iconografía (SVG vs Vector Icons): Se detectaron problemas de enlazado de fuentes (font linking) en compilaciones nativas de Android que provocaban que los íconos

de vector se renderizaran como caracteres corruptos. Para asegurar la portabilidad y evitar depender de fuentes externas a nivel sistema operativo, se rediseñó la iconografía inyectando trazados vectoriales puros (``Path``) utilizando ``react-native-svg``.

Monitoreo en Background (Background Fetch): Se implementó una tarea desatendida (``react-native-background-fetch``) que corre periódicamente. Por decisión de diseño, el test de Throughput se desactiva en background para no consumir masivamente el plan de datos 4G del usuario. Sólo se mide el Ping, Jitter, Pérdida y el RSSI de la antena.

Notificaciones de Degradación Local: Si la tarea en background detecta que la red está colapsada (Ping promedio > 300ms o Señal < -105 dBm), se utiliza ``@notifee/react-native`` para lanzar una notificación push local de alerta al usuario, cumpliendo con el requisito de monitoreo QoE preventivo.

Persistencia Reactiva (WatermelonDB): Se eligió WatermelonDB por sobre SQLite puro o Realm debido a su naturaleza asíncrona y basada en Observables (RxJS). Esto permite que el historial de cientos de mediciones se actualice en la pantalla en tiempo real sin bloquear el hilo principal ni provocar tirones en las animaciones de la UI.

UX/UI Consolidada (Mapa + Bottom Sheet): En lugar de aislar el mapa de calor y el listado histórico en pantallas diferentes, se diseñó una interfaz donde el mapa funciona como fondo interactivo y el listado se despliega sobre una lámina inferior deslizable (Bottom Sheet). Esto permite correlacionar inmediatamente el listado de métricas con su geolocalización.

Focalización en Android (APK): Dada la necesidad de extraer los datos de la potencia de la antena celular (RSSI) mediante el puente nativo y la API TelephonyManager de Android, el alcance del proyecto se cerró exclusivamente a compilar un entregable .apk. No es posible proveer binarios para iOS (.ipa) ni clientes de Expo Go puro, ya que Apple restringe fuertemente el acceso de bajo nivel a sus antenas celulares por privacidad de sistema operativo.

Limitaciones Conocidas

Timeouts en Redes Críticas (Subida 0 MB/s): En situaciones donde el RSSI de la antena es extremadamente bajo (ej. -117 dBm o inferior), el test de subida (que intenta enviar un payload hacia el backend) fallará silenciosamente por límite de tiempo (timeout), arrojando 0 MB/s. Esto no es un bug del motor de cálculo, sino un reflejo real de que el canal de subida de la celda está inutilizable.

Gestión Agresiva de Batería en Android/iOS (Doze Mode): El sistema de muestreo periódico en background solicita ejecutarse cada 15 minutos, pero la frecuencia real de ejecución queda delegada al sistema operativo. En celulares con "Ahorro de batería" estricto (ej. Xiaomi, Huawei) las mediciones en segundo plano podrían pausarse hasta que el usuario desbloquee la pantalla.

Limitación en el Tunneling (Ngrok): El test de Throughput celular requiere conectarse al Backend local. Para ello, dependemos de túneles públicos gratuitos como Ngrok. Estos túneles imponen límites de ancho de banda. Si la velocidad de 4G da un resultado extrañamente bajo (ej. tope de 2 MB/s fijos), suele ser un cuello de botella artificial del propio túnel Ngrok gratuito, y no de la antena celular.